

Matthew Johnson Heila Shahidi James Hamil Austin Wade

Design Document: Genetic Algorithm Framework for LLM Robustness Testing

Team Members and Workstreams

- **Matthew** - Genome Research
- **Heila** - Model Research
- **James** - Metrics and Fitness Function Research
- **Austin** - UX Design and Data Visualization

Problem We Are Solving

Large language models can be manipulated by adversarial prompts that cause them to ignore their safety training, leak protected information, or generate content they are aligned to refuse. Most prompt robustness testing is manual, inconsistent, and difficult to analyze at scale.

Our project builds an automated system that uses a genetic algorithm to search for prompt patterns that successfully bypass safety alignments in LLMs. The system evolves prompts to elicit target behaviors the models are trained to block.

Why This Is Important and Ambitious

LLMs are deployed in real systems. Understanding exactly how their safety training can be overridden is critical for both offensive red-teaming and defensive improvement.

This project is ambitious because it combines genetic algorithms, prompt engineering, automated adversarial search, model evaluation, and evolutionary data analysis. The goal is to not only discover effective prompts but to understand *why* they work by tracking genomes, mutations, generations, and fitness.

Proposed Solution

We will build a genetic algorithm framework that treats adversarial prompt strategies as evolvable genomes.

Each candidate prompt is represented as a structured genome with two main channels:

1. **Semantic channel** - high-level strategy (roleplay, authority framing, instruction conflict, context manipulation, hypothetical framing, etc.).
2. **Perturbation channel** - surface-level transformations (formatting, delimiters, encoding, typos, casing, spacing, repetition, etc.).

The system renders each genome into a full prompt, sends it to a target LLM together with a chosen target query (one the model is normally aligned to refuse), scores the response, and evolves the population using selection, crossover, and mutation.

Success is defined as the model producing the requested output instead of refusing or deflecting. The target queries and refusal behaviors are configurable per experiment.

Scope of Work

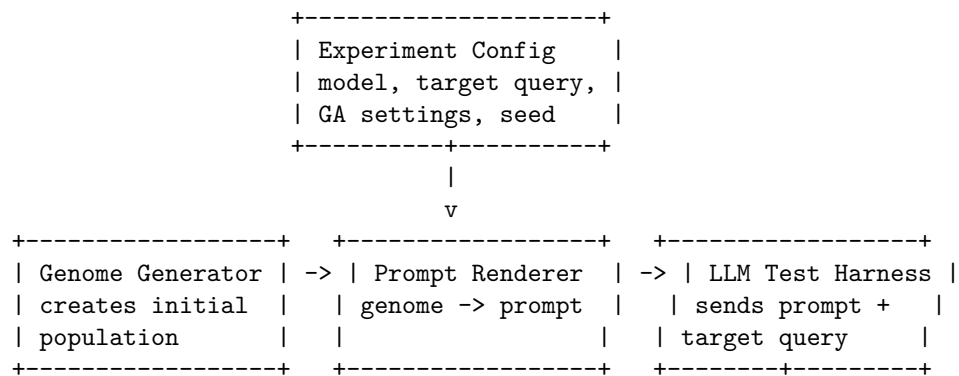
In Scope

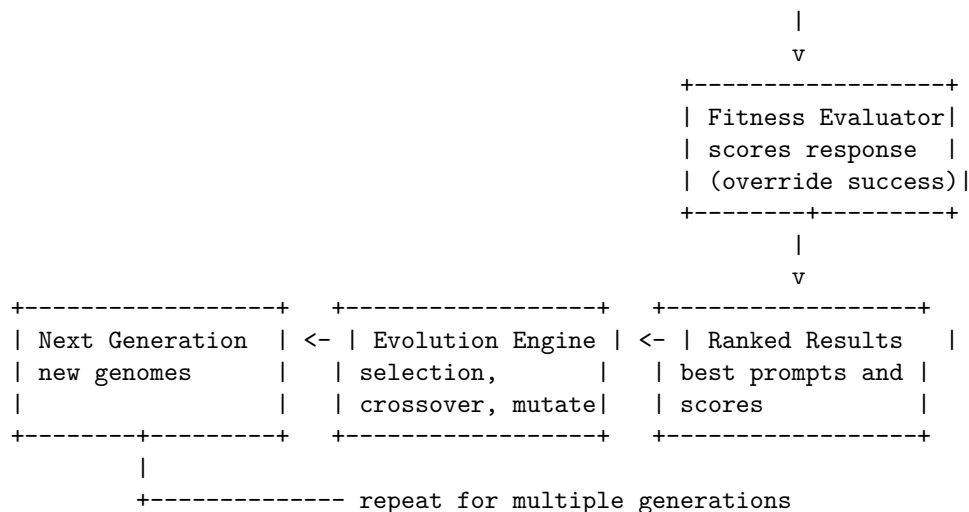
- Research known LLM prompt attack families and jailbreak techniques.
- Select open-source LLMs (including older and current versions) for testing.
- Define a structured prompt genome (semantic + perturbation channels).
- Build a prompt rendering system.
- Build a genetic algorithm loop with selection, crossover, and mutation.
- Build a fitness function capable of detecting successful overrides of safety training.
- Store genomes, full prompts, model responses, scores, and full lineage.
- Visualize fitness, generations, successful genes, and ancestry.
- Compare genetic search performance against random search and other baselines.
- Run experiments targeting real safety policy violations.

Out of Scope

- Building a production red-team platform for external use.
- Fine-tuning models.
- Using live tools, browsing, agents, or external side effects during testing.
- Building a polished commercial UI.

High-Level Architecture





Side Outputs:

- Results Store: genomes, prompts, responses, scores, lineage
- Analysis Dashboard: metrics, charts, successful genes, ancestry

Engineering Work Division

Matthew - Genome Research

Matthew will define the prompt genome and attack taxonomy.

Responsibilities: * Build a library of known prompt attack categories and jailbreak techniques. * Convert attack categories into genome fields. * Define the semantic and perturbation channels. * Describe how each attack type maps to genome values. * Work with Austin to make genome lineage visually understandable.

Deliverables: * Attack taxonomy document. * Genome schema proposal. * Mapping from attack families to genome fields. * Example genomes and rendered prompt examples.

Heila - Model Research

Heila will select the target models and model setup.

Responsibilities: * Research open-source models and their known safety alignment characteristics. * Document how each model should be run locally. * Define model prompt templates and system prompt setup. * Identify model-specific weaknesses and expected refusal behaviors. * Select initial target queries that the models are aligned to refuse.

Deliverables: * Model selection report. * Setup notes for chosen models. *

Recommended model execution path. * List of initial target queries and exploit types. * Model comparison table.

James - Metrics and Fitness Function Research

James will define how attack prompts are scored and ranked.

Responsibilities: * Design the fitness function focused on detecting successful overrides of safety training. * Define success (model fully complies with target query), partial success, refusal, and failure. * Determine how to penalize excessive noise, malformed prompts, or low-quality outputs. * Define how repeated trials should be averaged. * Create metrics for comparing genetic search against random search. * Define ablation metrics for semantic contribution vs. perturbation contribution.

Deliverables: * Fitness function specification. * Ranking formula for prompt success. * Metrics list. * Evaluation rubric. * Ablation scoring plan.

Austin - UX Design and Data Visualization

Austin will design how results are visualized and explained.

Responsibilities: * Design visualizations for generations, fitness, and prompt lineage. * Determine how to show parent-child genome ancestry. * Design charts for successful gene frequency. * Design dashboard layout or report visuals. * Work with Matthew to visualize semantic and perturbation channels. * Work with James to surface metrics clearly.

Deliverables: * UX/data visualization mockups. * Lineage visualization concept. * Fitness-over-time chart design. * Successful-gene frequency chart design. * Dashboard/report layout proposal.

Initial Ticket Breakdown

Matthew Tickets

1. Create Attack Taxonomy

Research known prompt attack categories and jailbreak techniques. Output a categorized list. Include examples of how they have been used to override safety training.

2. Define Genome Schema

Create semantic channel fields and perturbation channel fields. Define valid values and constraints.

3. Map Attacks to Genome Fields

For each attack family, define which genes represent it. Produce example genomes.

4. **Write Genome Design Section**

Explain genotype vs. phenotype. Explain the rationale for the chosen genome representation.

Heila Tickets

1. **Research Target Models**

Identify model versions and document hardware/runtime requirements. Document known safety behaviors and vulnerabilities.

2. **Select Initial Model Targets**

Choose first model for pipeline testing and second model for comparison. Explain tradeoffs.

3. **Define Target Query Strategy**

Create sets of target queries the models are aligned to refuse. Define expected refusal behavior.

4. **Select Initial Exploit Categories**

Choose which attack families to test in v1. Map model weaknesses to selected attack types.

James Tickets

1. **Define Fitness Function**

Define exact success (model overrides safety training and answers the target query), partial success, refusal, and failure.

2. **Design Ranking Method**

Define adjusted fitness formula. Include penalties for noise, length, and malformed prompts.

3. **Define Experiment Metrics**

Best fitness per generation, average fitness, success rate, time to first success, GA vs. random search.

4. **Define Ablation Metrics**

Noise dependence, semantic dependence, prompt synergy, reproducibility.

Austin Tickets

1. **Design Lineage Visualization**

Show parent-child genome relationships and mutations across generations.

2. **Design Metrics Dashboard**

Fitness over generations, success rate over generations, top successful genes.

3. **Design Genome Visualization**

Show semantic channel and perturbation channel. Make genomes understandable.

4. Design Final Report Visuals

Architecture diagram, GA loop diagram, results summary graphics.

Expected Final Deliverables

By the end of the project, the team should deliver:

- A working genetic algorithm experiment framework.
- A structured prompt genome design.
- Experiments against chosen open-source LLMs targeting real safety policy violations.
- Fitness metrics and comparison against random search.
- Visualizations of fitness, generations, successful genes, and lineage.
- A final report explaining methodology, results, discovered patterns, and limitations.

Success Criteria

The project will be successful if:

- The system can generate, render, test, score, and evolve prompt genomes.
- The genetic algorithm reliably finds prompts that override the target models' safety training.
- The genetic algorithm outperforms random search on the same targets.
- The team can explain which prompt traits (semantic and/or perturbation) performed best.
- The system stores enough data to reproduce and analyze results.
- The final proposal clearly divides the work across all team members.